

API Overview

Data Structures

- Note: Only the struct members annotated in this document are relevant for CPU/GPU usage. Fields without comments are NPU-specific and can be ignored when working with CPU/GPU delegates.

aml_config

```
typedef struct __aml_nn_config
{
    bool secure_config;
    int typeSize;
    int length;
    const char *path;           // Path to the model file
    const char *pdata;
    amlnn_model_type modelType; // Model type
    amlnn_nbg_type nbgType;
    assign_user_address_t inOut;
    aml_forward_ctrl_t forward_ctrl;
    aml_compiler_args_t compiler_args;
    aml_hw_flag_t hw_flag;      // Target hardware
    int timeout_ms;
    const char *on_path;
} aml_config;
```

amlnn_model_type

```
typedef enum _amlnn_model_ {
    CAFFE = 0,
    TENSORFLOW,
    TENSORFLOWLITE,           // Only TFLite models are supported on CPU/GPU
    DARKNET,
    ONNX,
    KERAS,
    PYTORCH,
    ADLA_LOADABLE,
    MEDEL_MAX
} amlnn_model_type;
```

aml_hw_flag_t

```
typedef enum __aml_hw_flag_t
{
    AML_HW_NPU = 1,
    AML_HW_GPU = 2, // GPU backend
    AML_HW_CPU = 3  // CPU backend
} aml_hw_flag_t;
```

nn_input

```
typedef struct __nn_input
{
    int typeSize;
    int input_index;          // Index of the input tensor, single input uses
0
    int size;                 // Size of the input buffer
    unsigned char* input;     // Pointer to the input buffer
    amlnn_input_type input_type; // Input buffer type
    input_info info;          // Input meta information
    int subgraph_index;       // Currently unused, keep 0
    int input_valid_length;
}nn_input;
```

amlnn_input_type

```
typedef enum _amlnn_input_ {
    RGB24_RAW_DATA = 0,
    TENSOR_RAW_DATA,
    QTENSOR_RAW_DATA,
    BINARY_RAW_DATA,          // CPU/GPU only support raw data; preprocessing
(normalization/quantization) must be done by user code
    INPUT_DMA_DATA,
    INPUT_DMA_SECURE_DATA,
    NV12_RAW_DATA,
    RAW_DATA_MAX
} amlnn_input_type;
```

input_info

```
typedef struct
{
    int valid;
    int int16_type;
    int preprocess_debug;
    float mean[INPUT_CHANNEL];
    float scale;
    aml_input_format_t input_format;
    aml_input_data_type_t input_data_type; // Actual data type passed to NN SDK
}input_info;
```

aml_input_data_type_t

```
typedef enum {
    AML_INPUT_U8,             // uint8 input to NN SDK
    AML_INPUT_I8,             // int8 input to NN SDK
    AML_INPUT_FP32,           // float32 input to NN SDK
} aml_input_data_type_t;
```

aml_output_config_t

```
typedef struct __amlnn_module_out_data_t
{
    int typeSize;
    aml_module_t mdType;
    aml_perf_mode_t perfMode;
    aml_output_format_t format; // Output buffer format
    aml_output_order_t order;
    aml_invoke_info_t invoke;
} aml_output_config_t;
```

aml_output_format_t

```
typedef enum {
    AML_OUTPUT_FORMAT_F32          = 0,
    AML_OUTPUT_FORMAT_F16          = 1,
    AML_OUTPUT_FORMAT_INT8         = 2,
    AML_OUTPUT_FORMAT_UINT8        = 3,
    AML_OUTPUT_FORMAT_INT16        = 4,
    AML_OUTPUT_FORMAT_UINT16       = 5,
    AML_OUTPUT_FORMAT_INT32        = 6,
    AML_OUTPUT_FORMAT_UINT32       = 7,
    AML_OUTPUT_FORMAT_INT64        = 8,
    AML_OUTPUT_FORMAT_UINT64       = 9,
    AML_OUTPUT_FORMAT_BOOL         = 10,
    AML_OUTPUT_FORMAT_UNDEFINED    = 11,
} aml_output_format_t;
```

aml_output_order_t

```
typedef enum {
    AML_OUTPUT_ORDER_NHWC = 0,
    AML_OUTPUT_ORDER_NCHW = 1,
} aml_output_order_t;
```

aml_output

```
typedef struct amlnn_output {
    unsigned int typeSize;
    unsigned int output_num; // Number of output tensors
    outBuf_t out[MAX_OUTPUT_NUM]; // Output tensor buffers
} aml_output;
```

outBuf_t

```
typedef struct out_buf
{
    unsigned int size;           // Output buffer size
    char name[MAX_NAME_LENGTH];
    unsigned char *buf;         // Pointer to tensor buffer
    nn_buffer_params_t *param;
    aml_output_format_t out_format;
    int output_valid_length;
} outBuf_t;
```

tensor_info

```
typedef struct {
    unsigned int valid;
    unsigned int num;           // Number of tensors
    info_t *info;               // Pointer to tensor info array
} tensor_info;
```

info_t

```
typedef struct {
    unsigned int dim_count;
    unsigned int sizes_of_dim[MAX_TENSOR_NUM_DIMS]; // Tensor shape
    unsigned int data_format;                       // See nn_buffer_format_e
    unsigned int data_type;
    unsigned int quantization_format;
    int fixed_point_pos;
    float TF_scale;                                // Quantization scale
    int TF_zeropoint;                              // Quantization zero point
    char name[MAX_NAME_LENGTH];
} info_t;
```

nn_buffer_format_e

```
typedef enum _nn_buffer_format_e
{
    NN_BUFFER_FORMAT_FP32      = 0, // float32
    NN_BUFFER_FORMAT_FP16      = 1,
    NN_BUFFER_FORMAT_UINT8     = 2, // uint8
    NN_BUFFER_FORMAT_INT8      = 3, // int8
    NN_BUFFER_FORMAT_UINT16    = 4,
    NN_BUFFER_FORMAT_INT16     = 5,
    NN_BUFFER_FORMAT_INT32     = 8,
    NN_BUFFER_FORMAT_INT64     = 10,
    NN_BUFFER_FORMAT_BOOL      = 11,
} nn_buffer_format_e;
```

Functions

1. aml_module_create

```
void* aml_module_create(aml_config* config);
```

- **Purpose:** Load the model.
- **Parameters:** `config` – initialization configuration.
- **Return value:** On success, returns a context pointer; otherwise returns `NULL`.

2. aml_module_destroy

```
int aml_module_destroy(void* context);
```

- **Purpose:** Unload the model.
- **Parameters:** `context` – context pointer.
- **Return value:** `0` on success; non-zero on failure.

3. aml_module_input_set

```
int aml_module_input_set(void* context, nn_input *pInput);
```

- **Purpose:** Provide input data for inference.
- **Parameters:**
 - `context` – context pointer.
 - `pInput` – input configuration and data buffer.
- **Return value:** `0` on success; non-zero on failure.

4. aml_module_output_get

```
void* aml_module_output_get(void* context, aml_output_config_t outconfig);
```

- **Purpose:** Execute inference and obtain outputs.
- **Parameters:**
 - `context` – context pointer.
 - `outconfig` – inference configuration.
- **Return value:** On success, returns an output object; otherwise returns `NULL`.

5. aml_util_getTensorInfo

```
int aml_util_getTensorInfo(void *context, const char* model_data, tensor_info** in_tInfo, tensor_info** out_tInfo);
```

- **Purpose:** Query tensor metadata for model inputs and outputs.
- **Parameters:**
 - `context` – context pointer.
 - `model_data` – unused, pass `NULL`.
 - `in_tInfo` – pointer to the input tensor info pointer.
 - `out_tInfo` – pointer to the output tensor info pointer.
- **Return value:** `0` on success; non-zero on failure.

6. aml_util_freeTensorInfo

```
int aml_util_freeTensorInfo(tensor_info* tinfo);
```

- **Purpose:** Release tensor info returned by `aml_util_getTensorInfo`.
- **Parameters:** `tinfo` – tensor info pointer to free.
- **Return value:** `0` on success; non-zero on failure.
- **Note:** Always call this function after successfully obtaining tensor metadata to avoid memory leaks.